Synchronisationskontrolle - Paxos vs. 2PC

Problemstellung:

Ausgangslage: Transaktion in einem verteilten System mit mehreren beteiligten Datenbanken
Problem: Transaktion könnte während des Prozesses abgebrochen oder nur teilweise ausgeführt werden (-> Inkonsistente Daten/Datenverlust)
Erkenntnis: Der einzelne Knoten kann seine Daten nicht einfach ändern oder dem Client anzeigen, ohne dass die gesamte Aktion vorher von allen beteiligten Systemen bestätigt wurde.

--> Aktion muss synchronisiert werden
--> Gewährleistung der Konsistenz (Die gewünschte Aktion wird entweder überall durchgeführt, oder nirgendwo -> ***Atomar***)

2-Phasen-Commit-Protokoll (2PC)

Algorithmus im Datenbankmanagement -> stellt sicher, dass die Änderung atomar erfolgt ("Alles oder nichts")
Aufteilung in 2 Phasen:

1. Vorbereitungsphase
    - Koordinator fragt alle Komponenten nach Bereitschaft
    - Die Komponenten prüfen, ob sie genug interne Ressourcen haben, um die Aktion durchzuführen, präparieren sie, und melden Bereit oder Nicht Bereit
2. Commit Phase
    - Koordinator wertet Ergebnisse des Ready Checks aus
        - Alle Ready? -> Commit Signal
        - Mindestens 1 nicht ready -> Abort Signal
    - Die Datenbanken führen die entsprechende Aktion aus
        - Commit = Änderung ausführen -> Bestätigung an Koordinator
        - Abort = Rollback der Änderungen -> Bestätigung an Koordinator

Beispiele:

1. Überweisung von Bank A an Bank B
2. Onlineshop Bestellung mit verteiltem Lagerhaltungssystem
3. Competitive Online Game mit Matchmaking

Paxos

Konsensusprotokoll -> erzielt gemeinsames Ergebnis in einer Gruppe von Teilnehmern
Gewährleistung der Robustheit des Ergebnisses trotz eventueller Ausfälle von Systemen
Voraussetzungen von Konsens:

1. Correctness - Nur ein einziger Wert wird gewählt.
2. Liveness - Ein Wert wird gewählt, solange die Mehrheit der Teilnehmer verfügbar sind.
3. Safety - Nur ein vorgeschlagener Wert kann gewählt werden.

Übergang von 2PC zu Paxos:

Problem von 2PC -> Liveness Requirement verletzt, sobald 1 Teilnehmer ausfällt
siehe [Abb. 1]
Aber -> um eine eindeutige Entscheidung zu treffen genügt die **Mehrheit**
siehe [Abb. 2]
Mit der bloßen Anwendung des Mehrheitsprinzips erreicht man zwar eine Verbesserung zum reinen 2PC, Ausfälle des Koordinators führen allerdings weiterhin nicht zum Ziel.
-> Der Koordinator ist immernoch zuständig für die **Single source of truth** unter den Komponenten. Er kontrolliert die finale Entscheidung.

Bsp.: Der Koordinator crasht kurz bevor die Commit Phase durchgeführt wird.
-> Die Mehrheit der Teilnehmer ist in diesem Moment bereit eine Value zu akzeptieren.
--> Die Information über die final gewählte Value crasht jedoch mit dem Koordinator
---> Die übrigen Teilnehmer müssten ihr Wissen untereinander vergleichen, um ohne den Koordinator weiter arbeiten zu können
Um das Wissen zu Vergleichen, muss es lokal gespeichert werden - ein reines Bereit/Nicht Bereit reicht nicht mehr aus

Bsp.: Der Koordinator sendet die vorgeschlagene Value in der Vorbereitungsphase, anstatt nur nach Bereitschaft zu fragen.

- Sende `PREPARE "value"` an alle Teilnehmer
- Die Teilnehmer setzen `preparedValue` falls es noch nicht gesetzt wurde.
- Wenn die Mehrheit der Teilnehmer die gleiche Value prepared hat, sende ACCEPT mit der mehrheitlich gewählten Value an alle
- Akzeptiere die gewählte Value

Siehe [Abb. 3]
Der Koordinator ist in diesem Fall kein tatsächlicher Koordinator mehr, da er im Ausfall ersetzt wird durch das Wissen der übrigen Teilnehmer.
Ab diesem Beispiel also: Koordinator == Proposer
-> Nun können zwar Proposer Ausfälle toleriert werden, diese Handhabung ist jedoch nicht **korrekt**.
Wenn es bei Ausfällen oder anderweitigen Problemen nicht genug Informationen über die Mehrheit gibt, gibt es evtl. ein Unentschieden.
siehe [Abb. 4]

Zu einem spezifischen Zeitpunkt (bevor Proposer 1 ausfällt) gibt es ja aber eine Mehrheit.
Technisch gesehen ist es also nicht möglich einen anderen Wert zu Wählen. -> Der letzte "Commit" Schritt ist unnötig
Mit anderen Worten: Wenn die Mehrheit der Teilnehmer verspricht den vorgeschlagenen Wert anzunehmen, ist er bereits gewählt.
Beweis durch Widerspruch - These: "Ein Wert wird nur angenommen, wenn der Commit-Schritt durchlaufen wird"

siehe [Abb. 5]
Beide Werte werden akzeptiert -> Verletzt das Konsens Prinzip
Erkenntnis: Sobald die Mehrheit der Teilnehmer einen Wert prepared hat, soll dieser als gewählt betrachtet werden. Hierbei darf es zu keinem Unentschieden kommen.

-> Einführen von Metadaten - Proposal-ID
Bei einem Unentschieden mit bspw. `preparedValue=Adenauer, preparedProposalId=2` , `preparedValue=Kohl, preparedProposalId=1` würde somit einfach der Wert mit der höheren ID gewinnen. Der Wert mit der höheren ID darf ja aber nur gewinnen, wenn er auch die Mehrheit hat.
D.h. der konkrete Fall (`preparedValue=Adenauer, preparedProposalId=2` , `preparedValue=Kohl, preparedProposalId=1` , `preparedValue=Kohl, preparedProposalId=1`) **muss unmöglich sein**.
-> Reicht die Präsenz einer höheren Proposal-ID alleine aus, um das sicherzustellen?
Idee: Komponenten zunächst versprechen lassen, keine andere ID als "2" (die höchste) zu preparieren, um dann den zugehörigen Wert zu "2" zu versenden. => Verhält sich gleich zum 2PC = Nicht robust gegenüber Ausfällen
-> Besser: Nicht die höchste ID erzwingen, sondern **bevorzugen** - die Teilnehmer dürfen ihre `preparedValues` zugunsten einer Value mit höherer Proposal-ID ändern. (Die Mehrheit genügt ja!)
siehe [Abb. 6]

Die Erkenntnis vom vorherigen Schritt, dass ein Wert akzeptiert wird, sobald ihn die Mehrheit prepariert hat, fehlt hier nun.
Wenn Proposer-1 crasht, bevor die Accept-Nachricht an jemanden übermittelt wurde, kommt vielleicht Proposer-2 mit dem Vorschlag `preparedValue=Brandt, proposalId=3` . `Brandt` würde gewählt werden.

Wenn man nun diesen Schritt mit einfügt, erählt man den folgenden Ablauf:

1. Prepare-Phase
   - Proposer generiert `proposalId` und sendet sie an alle Teilnehmer.
   - Die Teilnehmer halten eine `minProposalId` . Wenn eine eintreffende `proposalId` höher ist, ersetze die `minProposalId` damit. Falls es bereits einen akzeptierten Wert gibt, antworte mit der `(acceptedProposalId, acceptedValue)` .
2. Accept-Phase
   - Proposer wartet auf Antwort von der Mehrheit der Teilnehmer. Wenn es `acceptedValues` gibt, wähle den Wert mit der höchsten `proposalId` und ersetze den eigenen Vorschlag damit. Sende ACCEPT an alle Teilnehmer.
   - Wenn ein Teilnehmer eine ACCEPT Nachricht mit `proposalId` `<` `minProposalId` erhält, verwerfe die Nachricht. Ansonsten, setze `acceptedProposalId` und `acceptedValue` .
3. Learn-Phase
   - Proposer: Wenn die Mehrheit den Vorschlag akzeptiert hat, ist der Wert gewählt.

siehe [Abb. 7]
Hierbei handelt es sich um den Paxos Algorithmus.

Dadurch, dass die Mehrheit der Teilnehmer in der Vorbereitungsphase indirekt versprochen haben, keinen niedrigeren Wert als die `minProposalId` zu akzeptieren, wird dieses Versprechen vom nächsten Proposer im Ausfall wieder aufgenommen und macht den Commit **sicher**. ##Vergleich Beide Protokolle wollen ein gemeinsames gleiches Ergebnis durch Konsens erhalten.
2PC wird eingesetzt wenn sich mehrere (evtl. non-equivalente) Teilnehmer einigen müssen, dass die

gewünschte Aktion sicher ausführbar ist.
Paxos wird eingesetzt, um Fehlertoleranz zu schaffen, während die Teilnehmer sich trotzdem auf einen einzelnen, korrekten Wert einigen.

Beispiel:
Sie buchen einen Urlaubstrip über eine Webseite, die Komplettpakete anbietet. Dort wählen Sie ein Hotel (Zimmer mit Doppelbett), einen Flug (mit 2 Sitzen nebeneinander, einer am Fenster), sowie einen Leihwagen für die Dauer des Aufenthaltes (mit 4 Sitzen und großem Kofferaum). Sie geben Ihre Zahlungsinformationen ein, und bestätigen den Kauf.
Der Kauf soll nur durchgeführt werden, wenn alle beteiligten Systeme ihre Buchung akzeptieren. Die Mehrheitsregel von Paxos ergibt in diesem Szenario keinen Sinn, wenn das Bankkonto die Kosten nicht decken kann, oder keine Zimmer im Hotel mehr frei sind, ist es irrelevant ob die Airline die gewünschten Sitze reservieren kann, oder ob der gebuchte Leihwagen verfügbar ist.
Die einzelnen Teilnehmer können jedoch mit dem Paxos Algorithmus umgesetzt sein. Die Bank darf die vorgemerkte Belastung des Kontos nicht vergessen, selbst wenn einzelne Fehler im Serversystem der Bank auftreten.

2-Phase-Commit-Protocol

Vorteile:
Atomarität: Datenkonsistenz
Koordination: Mehrere Teilnehmer einigen sich auf die vollständig sichere Durchführung einer Aktion.

Nachteile:
Mögliche Blockierungen und Haltezustände
Unflexibel
schlecht Skalierbar

Paxos Algorithmus

Vorteile:
Toleranz gegenüber Ausfällen
Flexibel für viele Konsens Szenarien, auch mit multiplen, sequenziellen Werten
Performance hoch skalierbar

Nachteile:
Komplex: schwierig zu Implementieren (mögliche Edge Cases)
Eventuell hoher Overhead, da viele Nachrichten versendet werden

Anhang

Abbildung 1:

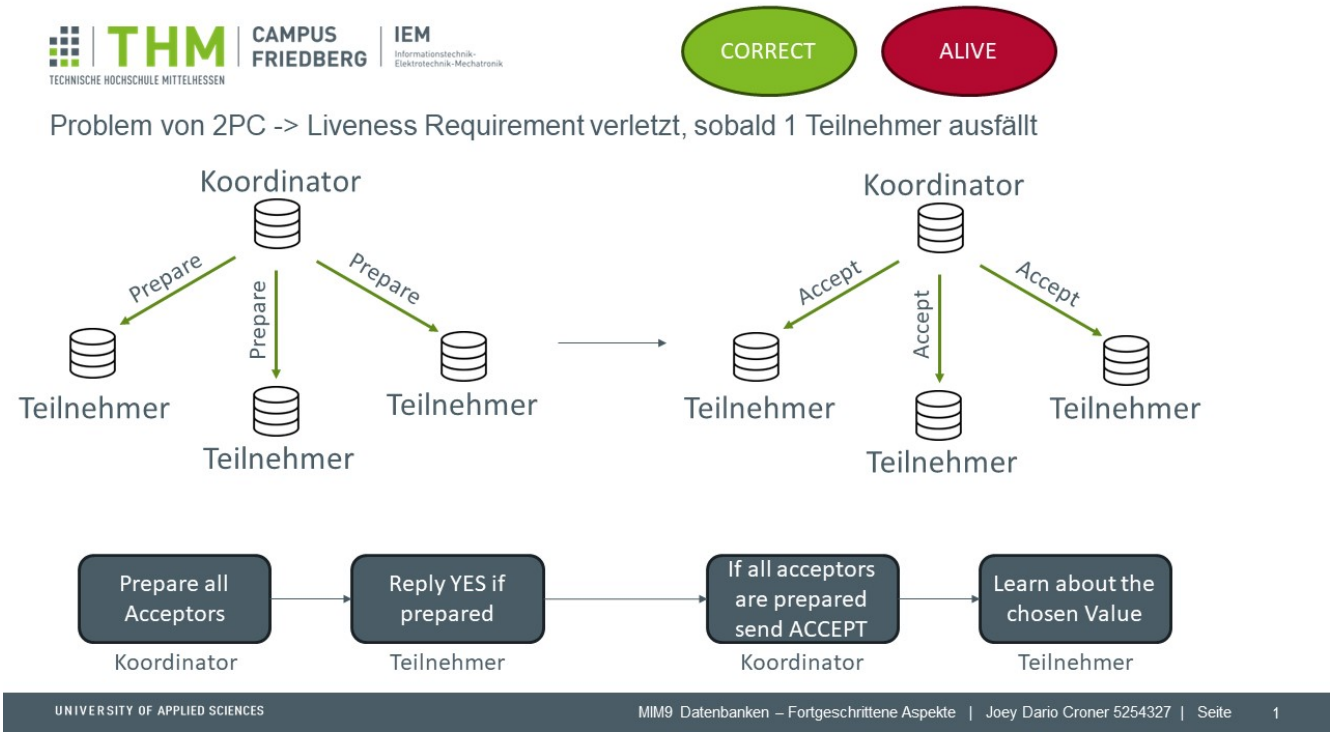


Abbildung 2:

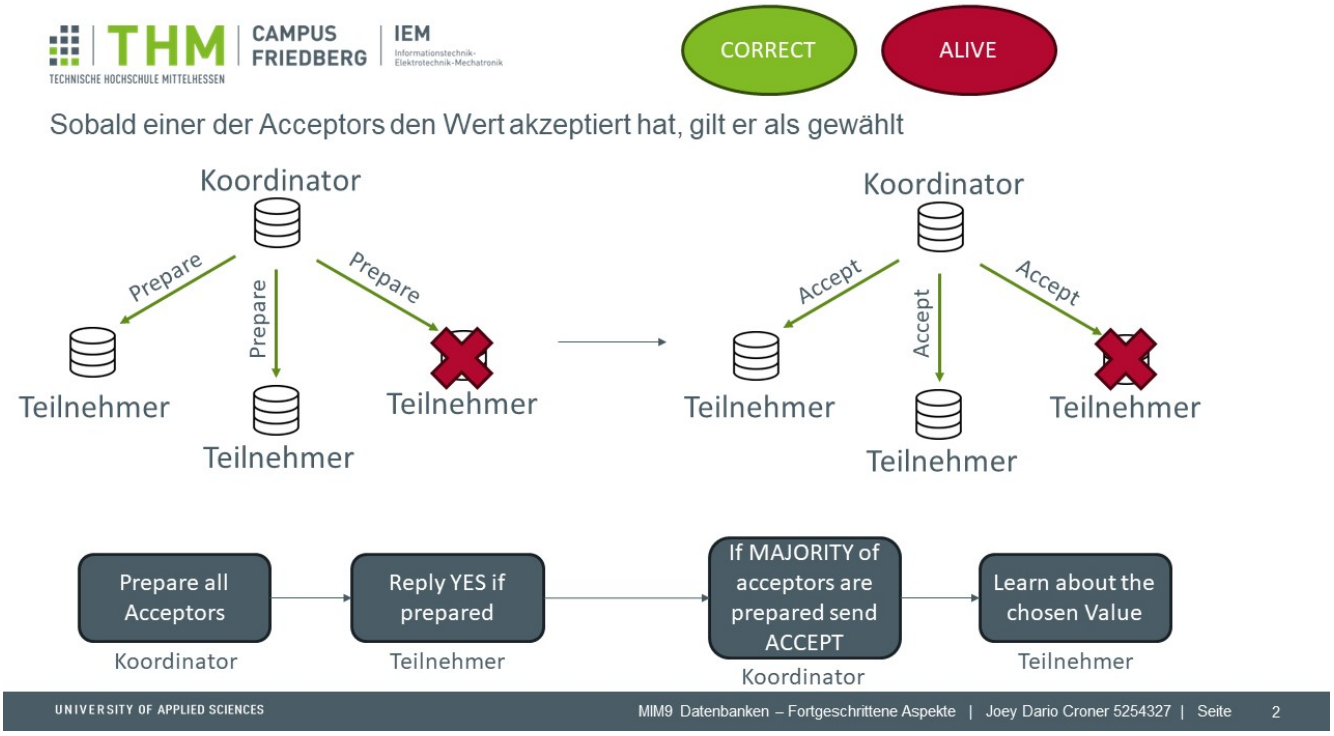
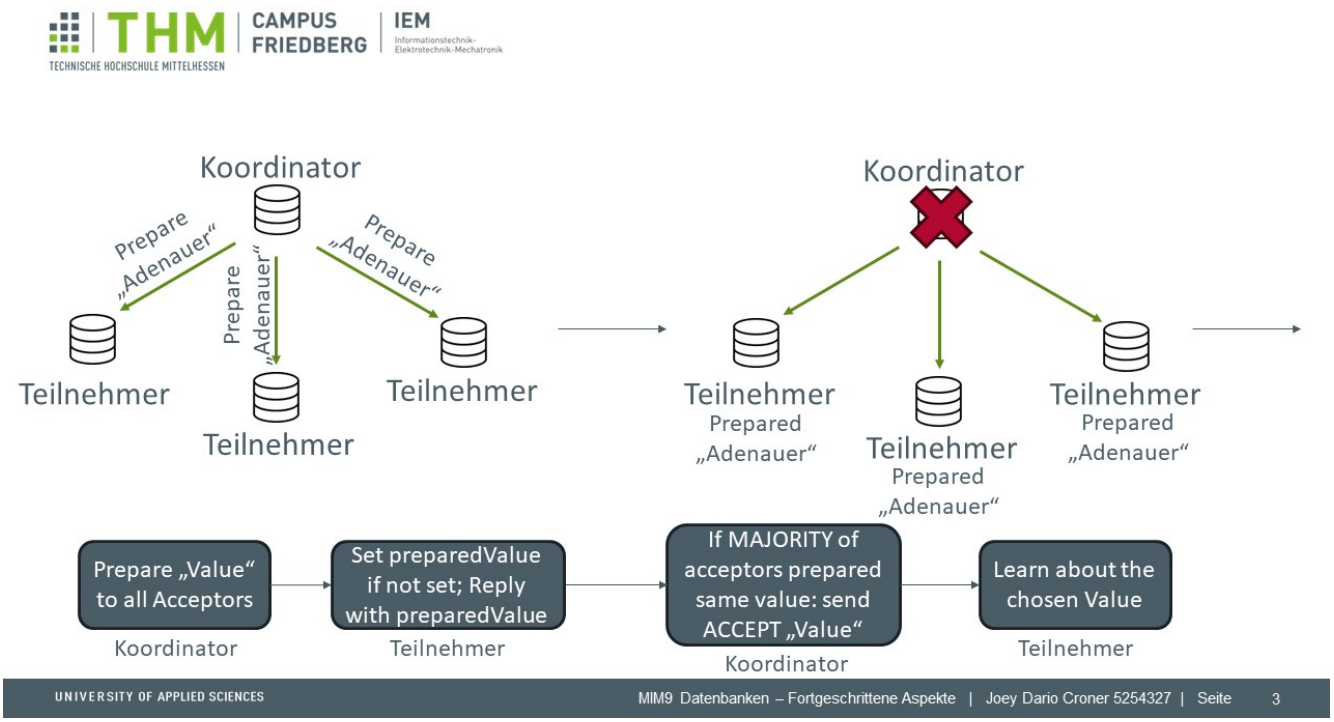


Abbildung 3:


THM
CAMPUS FRIEDBERG
IEM
Koordinator
Prepare „Adenauer“
Prepare „Adenauer“
Prepare „Adenauer“
Teilnehmer
Teilnehmer
Teilnehmer
Koordinator
Teilnehmer
Prepared „Adenauer“
Teilnehmer
Prepared „Adenauer“
Teilnehmer
Prepared „Adenauer“
Prepare „Value“ to all Acceptors
Koordinator
Set preparedValue if not set; Reply with preparedValue
Teilnehmer
If MAJORITY of acceptors prepared same value: send ACCEPT „Value“
Koordinator
Learn about the chosen Value
Teilnehmer
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 3

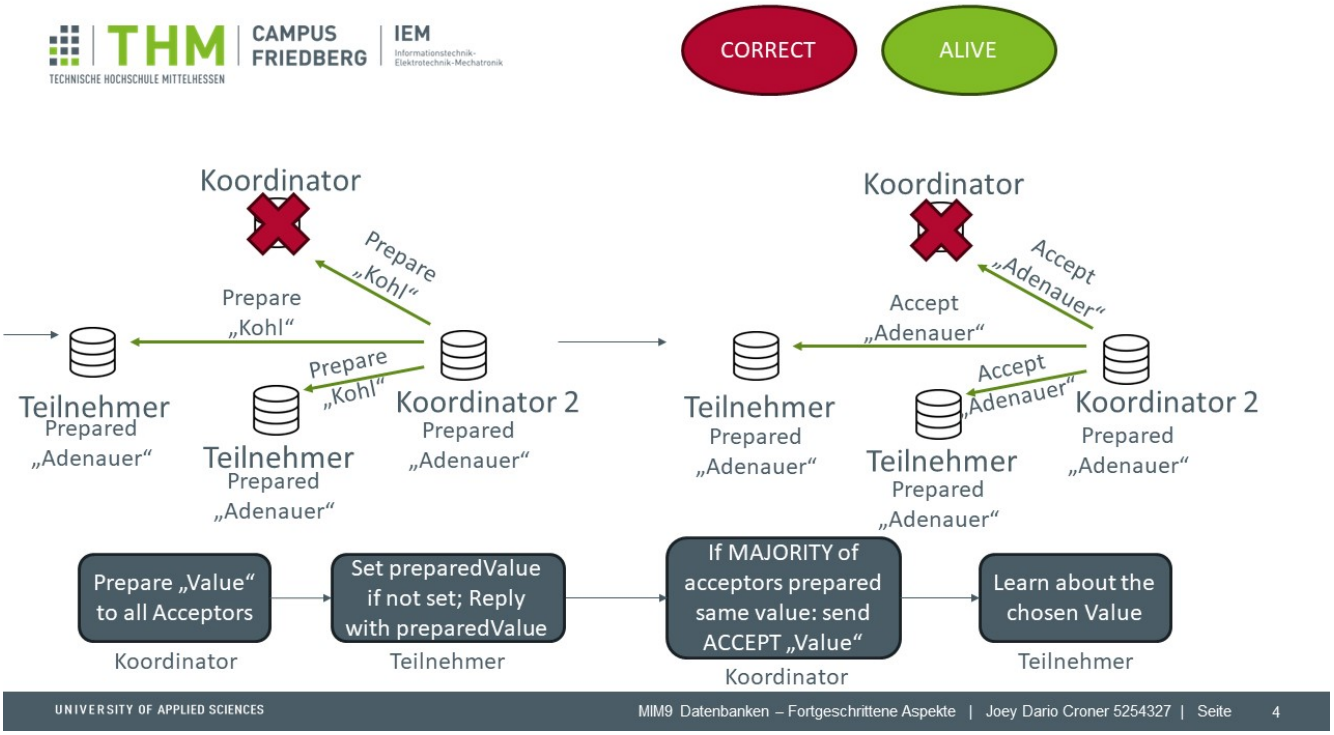
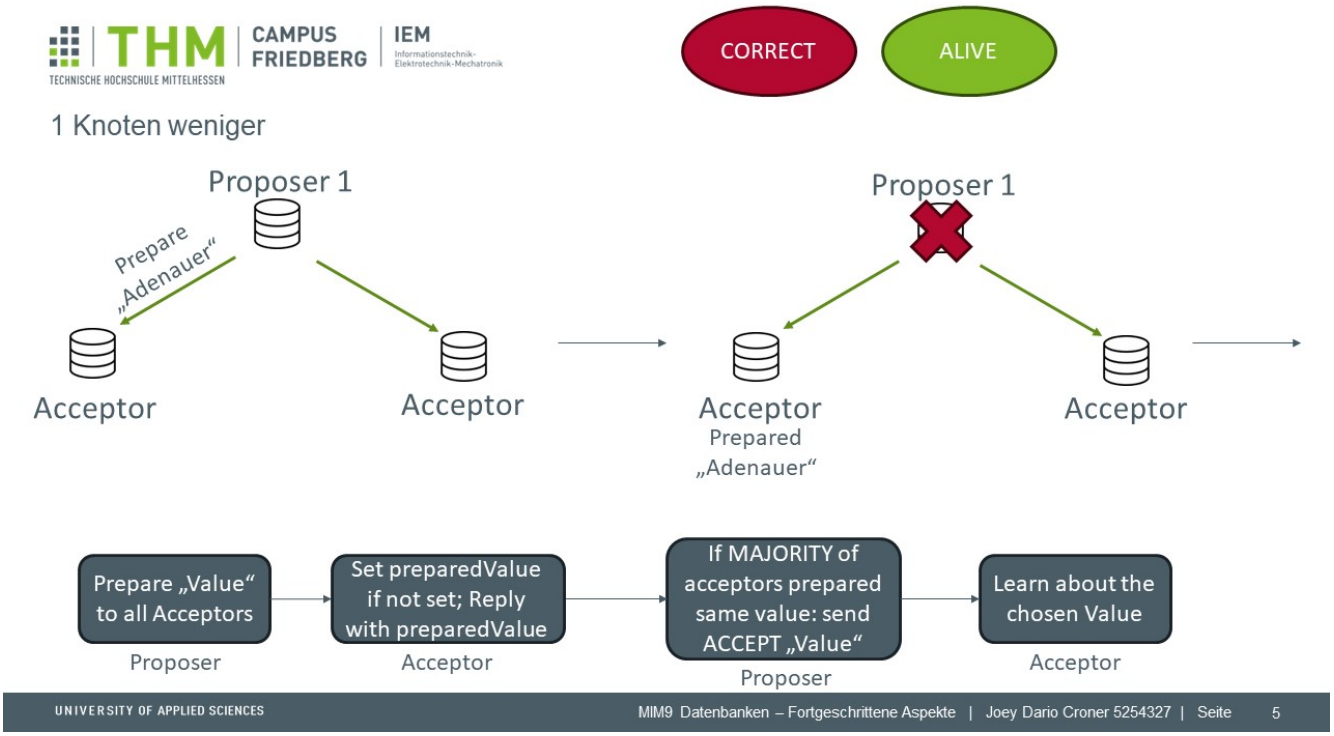

THM
CAMPUS FRIEDBERG
IEM
CORRECT
ALIVE
Koordinator
Prepare „Kohl“
Prepare „Kohl“
Prepare „Kohl“
Teilnehmer
Prepared „Adenauer“
Teilnehmer
Prepared „Adenauer“
Koordinator 2
Prepared „Adenauer“
Koordinator
Accept „Adenauer“
Accept „Adenauer“
Accept „Adenauer“
Teilnehmer
Prepared „Adenauer“
Teilnehmer
Prepared „Adenauer“
Koordinator 2
Prepared „Adenauer“
Prepare „Value“ to all Acceptors
Koordinator
Set preparedValue if not set; Reply with preparedValue
Teilnehmer
If MAJORITY of acceptors prepared same value: send ACCEPT „Value“
Koordinator
Learn about the chosen Value
Teilnehmer
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 4

Abbildung 4:


THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
CORRECT
ALIVE
1 Knoten weniger
Proposer 1
Prepare „Adenauer“
Acceptor
Acceptor
Proposer 1
Acceptor
Prepared „Adenauer“
Acceptor
Prepare „Value“ to all Acceptors
Proposer
Set preparedValue if not set; Reply with preparedValue
Acceptor
If MAJORITY of acceptors prepared same value: send ACCEPT „Value“
Proposer
Learn about the chosen Value
Acceptor
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 5

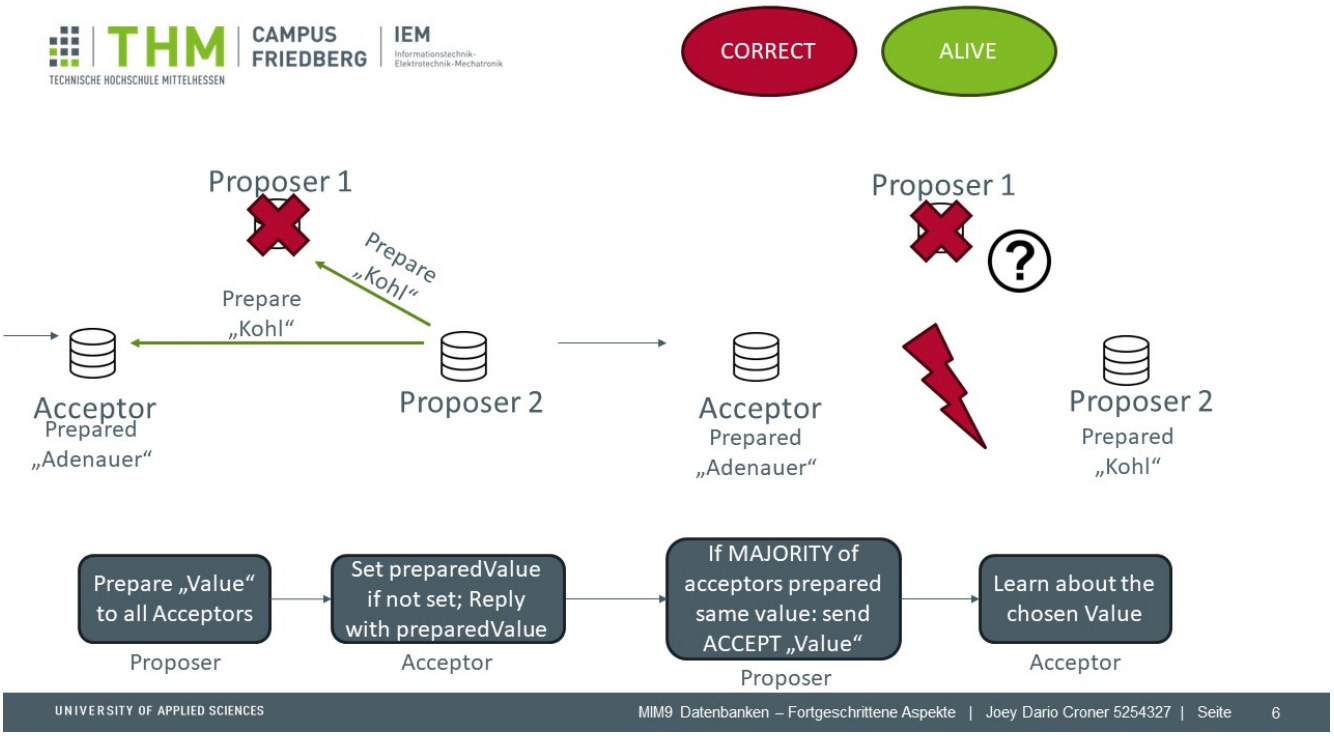
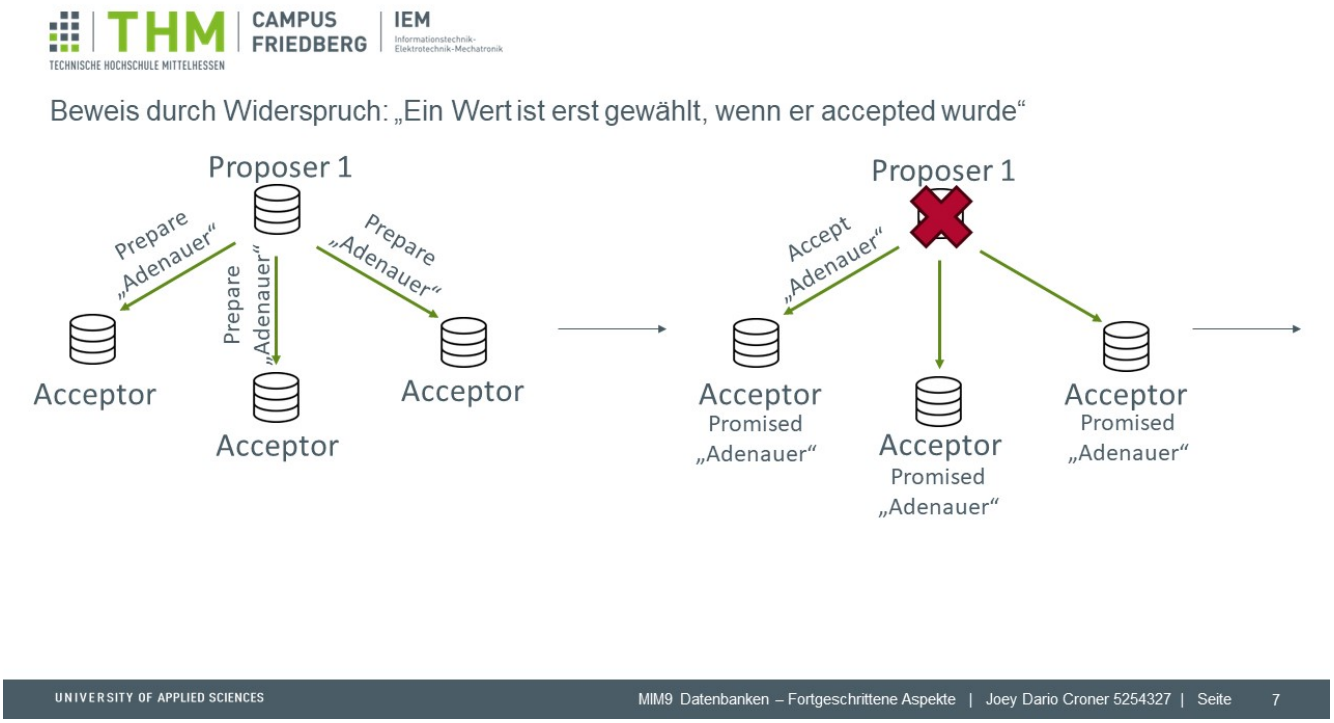

THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
CORRECT
ALIVE
Proposer 1
Prepare „Kohl“
Prepare „Kohl“
Acceptor
Prepared „Adenauer“
Proposer 2
Proposer 1
?
Acceptor
Prepared „Adenauer“
Proposer 2
Prepared „Kohl“
Prepare „Value“ to all Acceptors
Proposer
Set preparedValue if not set; Reply with preparedValue
Acceptor
If MAJORITY of acceptors prepared same value: send ACCEPT „Value“
Proposer
Learn about the chosen Value
Acceptor
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 6

Abbildung 5:


THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
Beweis durch Widerspruch: „Ein Wert ist erst gewählt, wenn er accepted wurde“
Proposer 1
Prepare „Adenauer“
Prepare „Adenauer“
Prepare „Adenauer“
Acceptor
Acceptor
Acceptor
Proposer 1
Accept „Adenauer“
Acceptor
Promised „Adenauer“
Acceptor
Promised „Adenauer“
Acceptor
Promised „Adenauer“
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 7

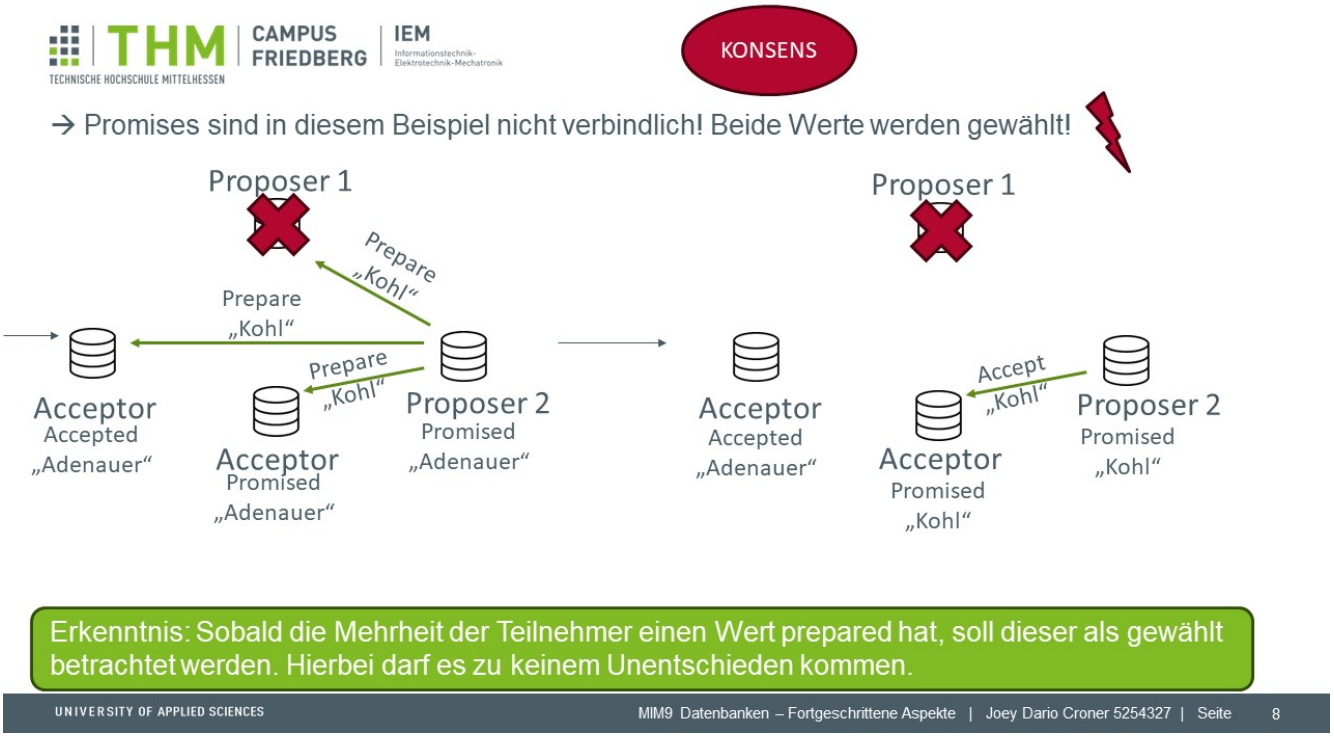
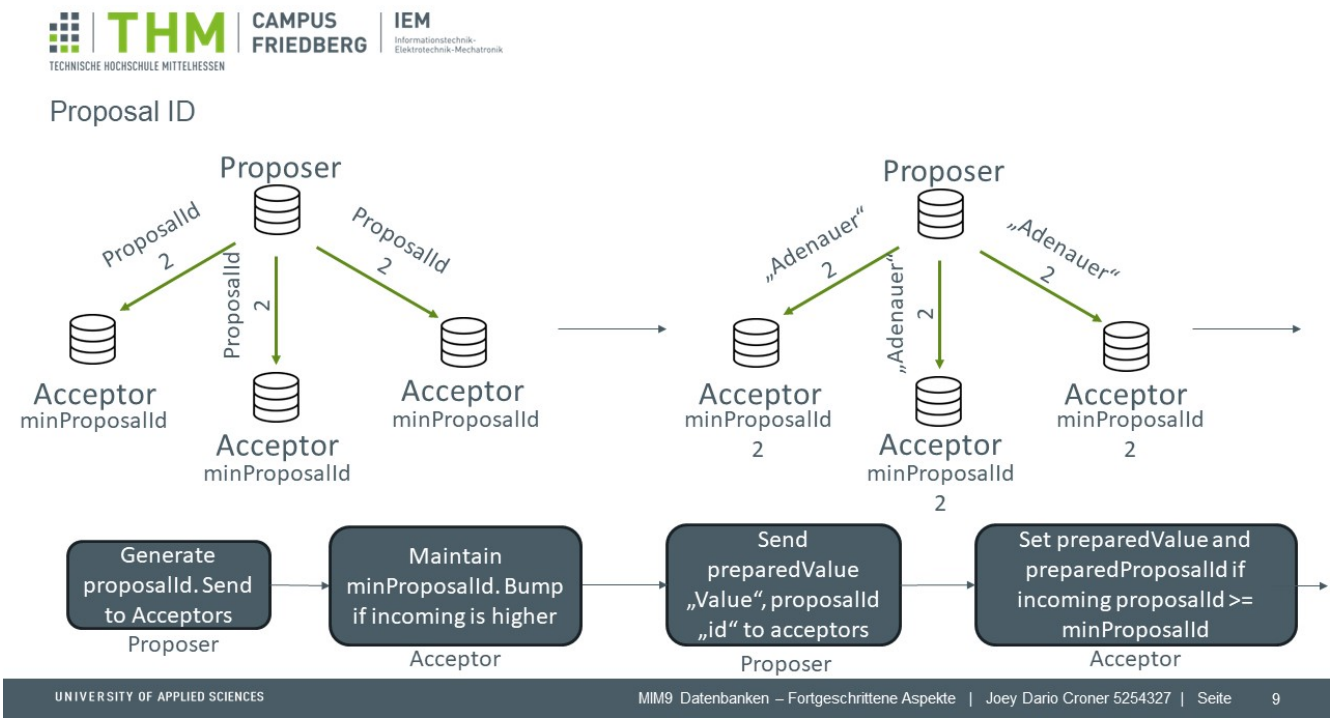

THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
KONSENS
→ Promises sind in diesem Beispiel nicht verbindlich! Beide Werte werden gewählt!
Proposer 1
Prepare „Kohl“
Prepare „Kohl“
Prepare „Kohl“
Acceptor
Accepted „Adenauer“
Acceptor
Promised „Adenauer“
Proposer 2
Promised „Adenauer“
Proposer 1
Acceptor
Accepted „Adenauer“
Accept „Kohl“
Acceptor
Promised „Kohl“
Proposer 2
Promised „Kohl“
Erkenntnis: Sobald die Mehrheit der Teilnehmer einen Wert prepared hat, soll dieser als gewählt betrachtet werden. Hierbei darf es zu keinem Unentschieden kommen.
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 8

Abbildung 6:


THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
Proposal ID
Proposer
ProposalId 2
ProposalId 2
ProposalId 2
Acceptor minProposalId
Acceptor minProposalId
Acceptor minProposalId
Proposer
„Adenauer“ 2
„Adenauer“ 2
„Adenauer“ 2
Acceptor minProposalId 2
Acceptor minProposalId 2
Acceptor minProposalId 2
Generate proposalId. Send to Acceptors
Proposer
Maintain minProposalId. Bump if incoming is higher
Acceptor
Send preparedValue „Value“, proposalId „id“ to acceptors
Proposer
Set preparedValue and preparedProposalId if incoming proposalId >= minProposalId
Acceptor
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 9

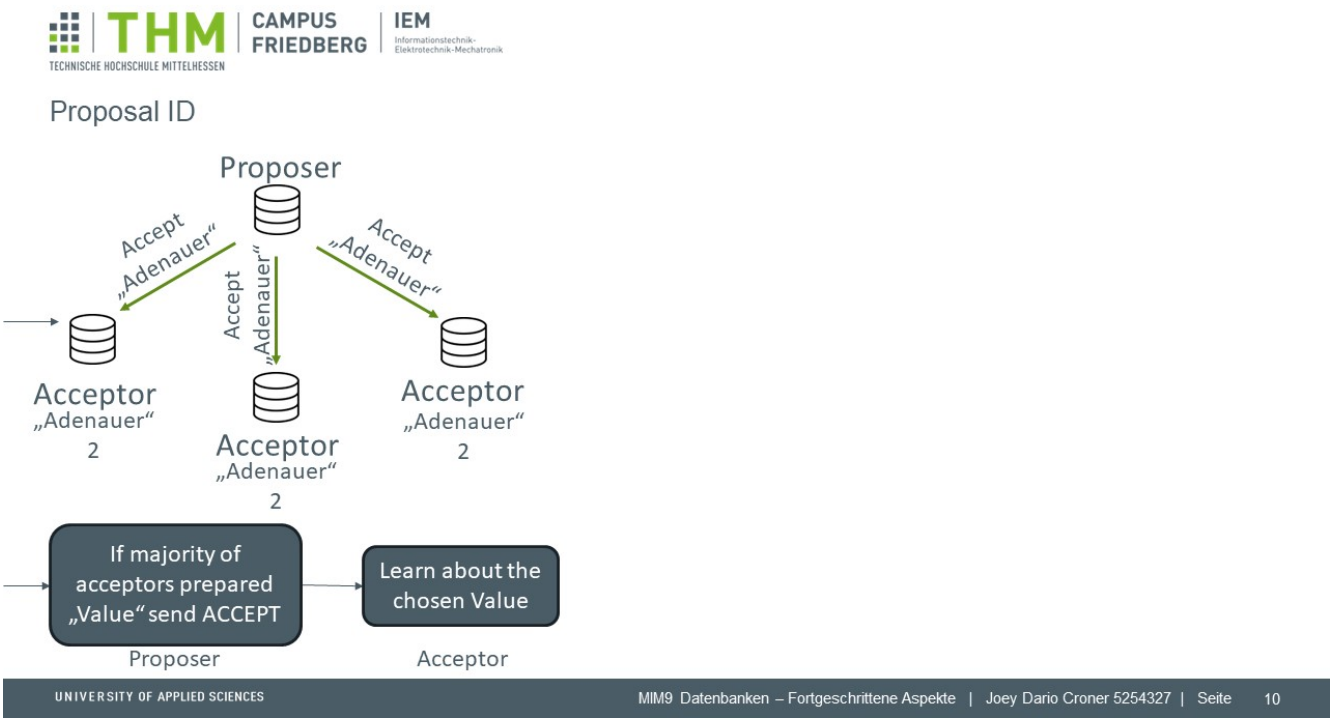
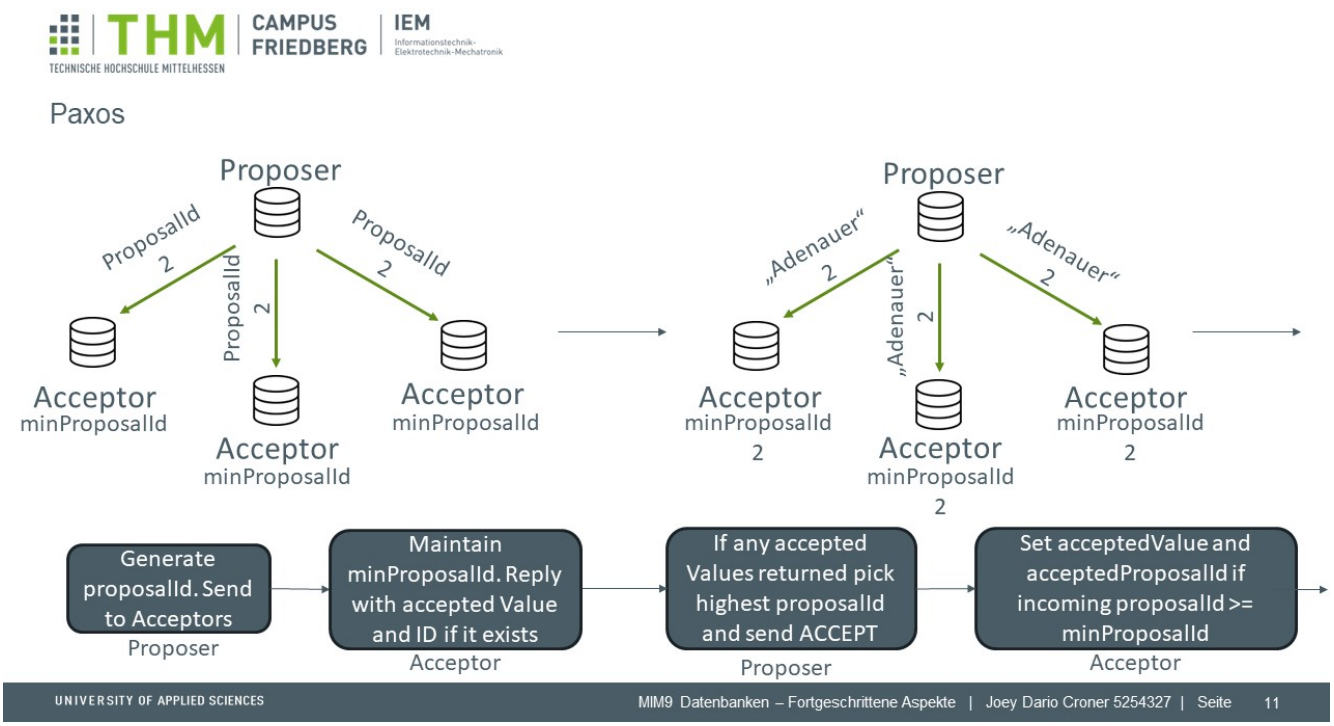
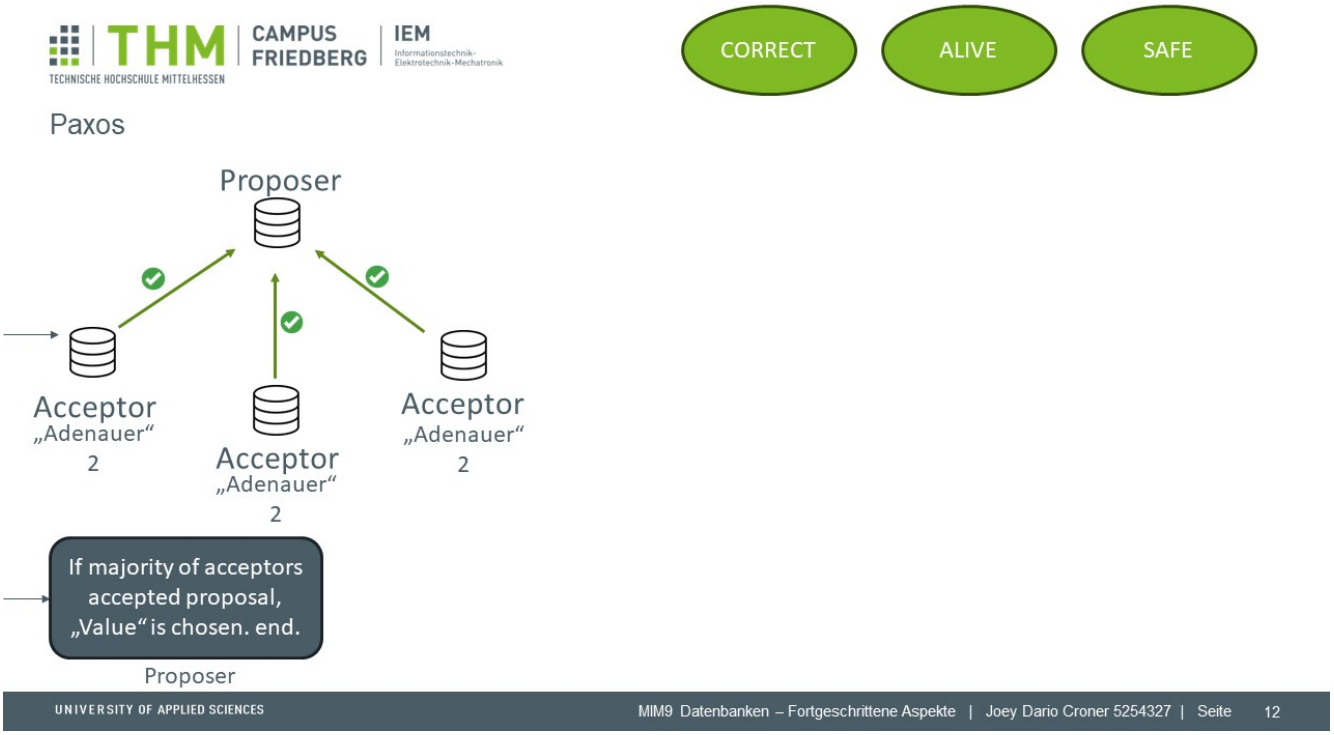

THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
Proposal ID
Proposer
Accept „Adenauer“
Accept „Adenauer“
Accept „Adenauer“
Acceptor „Adenauer“ 2
Acceptor „Adenauer“ 2
Acceptor „Adenauer“ 2
If majority of acceptors prepared „Value“ send ACCEPT
Proposer
Learn about the chosen Value
Acceptor
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 10

Abbildung 7:


THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
Paxos
Proposer
ProposalId 2
ProposalId 2
ProposalId 2
Acceptor minProposalId
Acceptor minProposalId
Acceptor minProposalId
Proposer
„Adenauer“ 2
„Adenauer“ 2
„Adenauer“ 2
Acceptor minProposalId 2
Acceptor minProposalId 2
Acceptor minProposalId 2
Generate proposalId. Send to Acceptors
Proposer
Maintain minProposalId. Reply with accepted Value and ID if it exists
Acceptor
If any accepted Values returned pick highest proposalId and send ACCEPT
Proposer
Set acceptedValue and acceptedProposalId if incoming proposalId >= minProposalId
Acceptor
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 11


THM
CAMPUS FRIEDBERG
IEM
TECHNISCHE HOCHSCHULE MITTELHESSEN
CORRECT
ALIVE
SAFE
Paxos
Proposer
Acceptor „Adenauer“ 2
Acceptor „Adenauer“ 2
Acceptor „Adenauer“ 2
If majority of acceptors accepted proposal, „Value“ is chosen. end.
Proposer
UNIVERSITY OF APPLIED SCIENCES
MIM9 Datenbanken – Fortgeschrittene Aspekte | Joey Dario Croner 5254327 | Seite 12